

Modernizing ops_pipeline3

From a Data Factory pipeline to a Spark notebook in Microsoft Fabric

Prepared for the customer · 2026-09-03

Chapter 1 – The ops_pipeline3 pipeline

Purpose. ops_pipeline3 is a **metadata-driven incremental ingestion** pipeline. It copies data from an **Eventhouse (KQL database)** into **Lakehouse Delta tables**, driven by a control table – so new source tables are onboarded by simply adding a row, with no change to the pipeline itself.

datacopyjobsetup → LookupDueJobs → ForEach → Copy (KQL → Lakehouse) → Delta tables

Parameters & variables

- ops_lakehouse – target Lakehouse (control table + destination tables).
- ops_kql_db – source KQL / Eventhouse database.
- vRunId – variable capturing the pipeline Run Id.

Step 1 – LookupDueJobs (Lookup)

Reads **every row** of dbo.datacopyjobsetup (firstRowOnly = false). Each row is one copy job with columns SourceName, WatermarkColumn, LastUpdated, DestinationName.

Step 2 – ForEach + CopyDynamicKQLTables

The **ForEach** loops over the lookup rows (batchCount = 20). Each row runs one **Copy**:

- **Source:** dynamic KQL <SourceName> | where <WatermarkColumn> > datetime(<LastUpdated>) (incremental pull).
- **Sink:** Lakehouse table dbo.<DestinationName>, **Append** (V-Order).

Note: the pipeline never updates LastUpdated; the notebook offers an optional step to advance it.

Chapter 2 – The replacement notebook

Markdown cells are shown as formatted text; code cells are shown as clean, readable code blocks with preserved indentation.

Replace **ops_pipeline3** with Spark

Metadata-driven incremental ingestion from an Eventhouse (KQL database) into Lakehouse Delta tables.

This notebook replaces the pipeline's two steps:

1. **LookupDueJobs** → read the control table **datacopyjobsetup**.
2. **ForEach + CopyDynamicKQLTables** → for each job, run an incremental KQL query and append the result to **<DestinationName>**.

Each control row provides: **SourceName**, **WatermarkColumn**, **LastUpdated**, **DestinationName**.

The KQL query built per job is: **<SourceName> | where <WatermarkColumn> > datetime(<LastUpdated>)**.

Tables are accessed by their **absolute OneLake path**, so no lakehouse needs to be attached to the notebook.

Parameters

```
# Eventhouse / KQL source (pipeline parameter ops_kql_db).
kql_cluster = "https://trd-7qm6ccfqm2rr8uzwff.z0.kusto.fabric.microsoft.com"
kql_database = "98405e07-5da7-48bd-a7c4-4fbfd276d4d1" # KQL database UID (GUID), not the
display name

# Lakehouse holding the control table and destination tables (pipeline parameter
ops_lakehouse).
workspace_id = "914d92b7-d202-4390-a2e2-0d7c3fcb3483"
lakehouse_id = "33e1f298-b3f3-4a67-ad30-732cb798d525" # ops_lakehouse artifact (GUID)

config_table = "datacopyjobsetup" # control table read by LookupDueJobs
dest_schema = "dbo" # schema in a schema-enabled lakehouse; set "" if not schema-enabled
max_parallel = 1 # sequential: Eventhouse export capacity is 1 (one .export at a time)
advance_watermark = False # pipeline does NOT advance the watermark; keep off to match it
```

Setup: OneLake paths, Kusto token, helpers

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

import notebookutils

_TABLES_ROOT = f"abfss://{workspace_id}@onelake.dfs.fabric.microsoft.com/{lakehouse_id}/Tables"

def table_path(table_name: str, schema: str = None) -> str:
    """Absolute OneLake path to a Delta table (works without an attached lakehouse)."""
    schema = dest_schema if schema is None else schema
    return f"{_TABLES_ROOT}/{schema}/{table_name}" if schema else f"{_TABLES_ROOT}/{table_name}"

# AAD token for the Eventhouse/Kusto cluster, using the notebook's identity.
kusto_token = notebookutils.credentials.getToken(kql_cluster)
```

```
def read_kql(query: str):
    """Run a KQL query against the Eventhouse and return a Spark DataFrame."""
    return (
        spark.read.format("com.microsoft.kusto.spark.datasource")
        .option("kustoCluster", kql_cluster)
        .option("kustoDatabase", kql_database)
        .option("kustoQuery", query)
        .option("accessToken", kusto_token)
        .option("readMode", "ForceDistributedMode") # export-based read, no 500K-row query cap
        .load()
    )

def build_query(source_name: str, watermark_column: str, last_updated) -> str:
    """Reproduce the pipeline's dynamic KQL: <source> | where <col> > datetime(<iso>)."""
    iso = last_updated.strftime("%Y-%m-%dT%H:%M:%SZ") if last_updated is not None else
    "1900-01-01T00:00:00Z"
    return f"{source_name} | where {watermark_column} > datetime({iso})"
```

Step 1 — LookupDueJobs (read the control table)

```
jobs = spark.read.format("delta").load(table_path(config_table)).collect()
print(f"Loaded {len(jobs)} copy job(s) from {table_path(config_table)}")
for j in jobs:
    print(f" - {j['SourceName']} -> {j['DestinationName']} (watermark {j['WatermarkColumn']} > {j['LastUpdated']})")
```

Step 2 — ForEach + Copy (incremental KQL → Lakehouse append)

```
def run_job(job) -> dict:
    source_name = job["SourceName"]
    watermark_column = job["WatermarkColumn"]
    last_updated = job["LastUpdated"]
    destination = job["DestinationName"]

    query = build_query(source_name, watermark_column, last_updated)

    last_err = None
    for attempt in range(5):
        try:
            df = read_kql(query)
            row_count = df.count()
            if row_count > 0:
                (
                    df.write.mode("append")
                    .format("delta")
                    .option("mergeSchema", "true")
                    .save(table_path(destination))
                )
            return {"source": source_name, "destination": destination, "rows": row_count,
"query": query}
        except Exception as exc: # noqa: BLE001 - retry transient Kusto throttling
            last_err = exc
            if "Throttled" in str(exc) or "throttling" in str(exc):
                time.sleep(min(60, 5 * 2 ** attempt)) # exponential backoff
```

```

        continue
    raise
raise last_err

results = []
with ThreadPoolExecutor(max_workers=max_parallel) as pool:
    futures = {pool.submit(run_job, j): j for j in jobs}
    for fut in as_completed(futures):
        job = futures[fut]
        try:
            res = fut.result()
            print(f"OK {res['source']} -> {res['destination']}: {res['rows']} row(s) appended")
            results.append(res)
        except Exception as exc: # noqa: BLE001 - report per-job failure, continue others
            print(f"FAIL {job['SourceName']} -> {job['DestinationName']}: {exc}")
            results.append({"source": job["SourceName"], "destination": job["DestinationName"],
                           "error": str(exc)})

print(f"\nCompleted {len(results)} job(s).")

```

Optional — advance the watermark

The original pipeline never updated **LastUpdated**, so re-runs re-copy the same rows. Enable this to make the run truly incremental by advancing each job's watermark to the max value just ingested.

```

from pyspark.sql import functions as F

if advance_watermark:
    from delta.tables import DeltaTable

    control = DeltaTable.forPath(spark, table_path(config_table))
    for res in results:
        if res.get("rows", 0) and "error" not in res:
            job = next(j for j in jobs if j["DestinationName"] == res["destination"])
            new_max = (
                spark.read.format("delta").load(table_path(res["destination"]))
                .agg(F.max(job["WatermarkColumn"]).alias("m"))
                .collect()[0]["m"]
            )
            if new_max is not None:
                control.update(
                    condition=F.col("DestinationName") == res["destination"],
                    set={"LastUpdated": F.lit(new_max)},
                )
                print(f"Watermark for {res['destination']} advanced to {new_max}")
        else:
            print("advance_watermark is False – matching original pipeline behavior (watermark unchanged).")

```

Chapter 3 – Parallelism & scaling

The ForEach used **batchCount = 20**. In Spark the real limit is the **Eventhouse** concurrency:

- **Single (query) mode** – query API, capped at **500,000 rows**. Small tables only.
- **Distributed mode** – reads via `.export t`. No row cap, but uses an export slot.

Why concurrency was limited to 1

The Eventhouse reported `Export Capacity: 1`, so 8 parallel jobs got **throttled**.

concurrent exports ≈ total cores × 0.75

So `max_parallel = 8` needs about **11 warm cores**.

Scaling on your F64 capacity

Open the **Eventhouse** → **Capacity Planner** and raise **Minimum consumption**; verify with `.show capacity` (Export row) before raising `max_parallel`.

max_parallel	Warm cores (~)	Action
1 (current)	any	nothing – sequential, safe on Capacity 1
4	~6	raise Minimum consumption 1–2 levels
8	~11	Minimum consumption to Medium/Large

Keeping more cores warm increases cost – raise for the batch window, then lower again. The notebook keeps a retry-with-backoff as a safety net for transient throttling.